SPRINGBOOT TESTING

Unit Testing : TaskService

we must first test the business logic inside TaskServiceImpl in complete isolation . Its dependency on TaskRepository is a challenge

Mocking : it is the most important theory for testing

Tools : We will use specific annotations from test environment @ExtendWith , @Mock @InjectMocks

Goal is to first test the methods inside TaskServiceImpl file

We want to verify that if we call the getTaskById(1L), it correctly calls the repository's findById(1L) method

TaskServiceImpl cannot exist without TaskRepository ,if we were to use a real TaskRepository in our test , it would connect to a real database .This breaks the rule of unit testing for two reasons :

its not isolated : our test would fail if db is down , even if java code in TaskServiceImpl is perfect .A unit test should only test the code in that one class

Q is how to test without a real repository

The solution to this is mocking . We create a fake simulated version of TaskRepository

a Mock is that simulated part , we create a fake TaskRepository that doesn't connect to any db.Its empty shell that we can programmatically control

we tell it , when your findById(1L) is called pretend you found a task and return this specific task object

Tools Junit5 and Mockito Annotations

under the *src/test/java/com/example/taskmanager* directory, create a new package named *service*. Inside this new package, create a new Java class called TaskServiceImplTest.java

```
@ExtendWith(MockitoExtension.class)
public class TaskServiceImplTest {

    @Mock
    private TaskRepository taskRepository;

    @InjectMocks
    private TaskServiceImpl taskService;
}
```

@ExtendWith(MockitoExtension.class) This is a Junit5 annotation .It tells the Junit test runner -> “before u run my tests pls initialise all the Mockito related stuff”

It is the magic that makes @Mock and @InjectMocks work

@Mock

private TaskRepository taskRepository

This is Mockito annotation that tells Mockito Create a fake simulated object of the TaskRepository class

@InjectMocks

private TaskServiceImpl taskService

It creates a real instance of our TaskServiceImpl class

fileds inside TaskServiceImpl requires taskRepository

It tries to inject any object marked with @Mock into it

result is

a taskService object that is real instance of TaskServiceImpl but its internal taskRepository field is populated with out **controllable** ,fake mock object

Writing the first test using universal Arrange-Act-Assert pattern , makes tests readable clean and easy to maintain

Arrange: Setting up all preconditions and input , how our mock should behave

Act: Call the method we are actually testing

Assert :Verify that the method returned expected outcome

```
@Test
void getTaskById_whenTaskExists_shouldReturnTask()
{
    //ARRANGE
    Task task=new Task(1L, "Test Task", "Description
for test", false);



when(taskRepository.findById(1L)).thenReturn(Optional.
of(task));
    //ACT

    Optional<Task>
foundTaskOptional=taskService.getTaskById(1L);
    //ASSERT
    assertThat(foundTaskOptional).isPresent();

assertThat(foundTaskOptional.get().getId()).isEqualTo(
1L);

assertThat(foundTaskOptional.get().getTitle()).isEqual
To("Test Task");

}
```

```
@Test
void getTaskById_whenTaskExists_shouldReturnTask()
{
    //ARRANGE
    Task task=new Task(1L, "Test Task", "Description for
test", false);
}
```

Task task=new Task(….) : We simply create a plain Java object to represent the data we expect to work with .This is our expected result

when(taskRepository.findById(1L)).thenReturn(Optional.of(task));

when :we say -> I am about to tell you the rule for the mock object

taskRepository.findById(1L) : specific method for which we are setting the rule for, we are testing the service and not the repository

it is just placeholder to identify the code

since taskService will be using taskRepository we need to set behaviour for taskRepository right

so whenever taskService executes a method that in turn calls taskRepository cause I tneeds it , but here we are using a fake taskRepository so we need to set that certain simulated behaviour for it right ?

.thenReturn(Option.of(Task) : this is the instruction we are telling mockito when you see that findById(1L) method being called on the mock , you must return an Optional containing our sample task object

ACT

Here we call the getTaskById method on our real taskService object .Internally this method will call taskRepository.findById(1L) .Since our service is using the mock repository , this will trigger rule we defined in arrange step and return the Optional<Task> we just defined above for it

ASSERT

assertThat(…) using a library called AssertJ whoch provides these clean redable assertation methods

we first say

assertThat(foundTaskOptional).isPresent() : to check if Task object from the Optional returned by service is not empty

assertThat(foundTaskoptional.get().getId()).isEqualTo(1L); we get task object from the Optional and assert that its ID is indeed 1L.

assertThat(foundTaskOptional.get().getTitle()).isEqualTo("Test task");

when we use any we meant to say any object that is of the type Task

```
when(taskRepository.save(any(Task.class))).thenAnswer(
invocation -> invocation.getArgument(0));
```

it gets an existingTask from the database (our mock service provides this)

it then changes the fields of that existingTask object in memory

finally it calls taskRepository.save() and passes this modified existingTask object to it

in update block we provide an existing object and it must return to us the updated the object but this is only the arrange phase right now , this is the present .in the future the update will be made

the update made in future will happen in the act block

we still need to define behavior of the arrange block for when it needs to return the updated block that hasn't been created yet

solution is thenAnswer – a dynamic response

.thenAnswer(invocation->invocation.getArgument(0))

whatever we say to a parrot it will say the same thing back to us analogy

invocation->invocation.getArgument(0)

invocation : special object that Mockito gives us ,it represents the entire method call that just happened .it contains details like method name save , the object it was called on taskRepository and the arguments that were passed to it

-> java std lambda expression separating input from the logic

invocation.getArgument(0)

core instruction

from that snapshot of method call get the first argument .since save method only passes the existing task as save(Task task) that service passed in

when taskRepository object is called to find by id of 1L it finds the existing task

when taskRepository object is called to save any object of class Task then return the same object it was called to save upon

the test then tests the method defined in the TestServiceImpl class thatis this

```
@Override
public Task updateTask(Long id, Task taskDetails) {
    // Find the existing task or throw an exception
    Task existingTask = taskRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Task not
found with id: " + id));

    // Update the fields
    existingTask.setTitle(taskDetails.getTitle());
    existingTask.setDescription(taskDetails.getDescription());
    existingTask.setCompleted(taskDetails.isCompleted());

    // Save the updated task
    return taskRepository.save(existingTask);
}
```

the three lines where existingTask is updated are important those are the ones that are tested

and after the arrange and ask block is done executing

the assertation phase checks if the results are the one we desired

so in this case the updated task we defined in test isn't saved over here the work is done by TaskServiceImpl class

infact it is actually a component or object that we are using to compare with our original one

showing that this is what is expected and this was what we got

---

the main purpose of this test is to verify the three lines of business logic inside the updateTask method:

inside TaskServiceImpl:

existingTask.setTitle(taskDetails.getTitle());

existingTask.setDescription(taskDetails.getDescription());

existingTask.setCompleted(taskDetails.isCompleted());

ABOVE tests work for happy path scenario

Making a test for unhappy path scenario

in our original class we use exceptions , so we must make sure in test when testing for unhappy path scenario the exception is being thrown we can use assertThrows for this

in all tests verify is used

During testing we check 2 main things

State Testing : this checks the outcome or the result

it means after the method ran , what is the final state , we use assertThat for testing this , to check if a return value is correct ,if field has changed .its like checking if u arrived at correct destination

Behaviour testing : checking if correct steps were followed and interacted with dependencies

1>in 1 st case

verify(taskRepository).save(taskToSave);

it is actually shorthand for

verify(taskRepository,times(1)).save(taskToSave);

when we don't verify the number of times Mockito assumes it means you want exactly one

Was the save method ever called on the taskRepository mock

if it was was the object passed to it .equals() the taskToSave object

2) verifying a method was never called

this is crucial for handling testing error handling and conditional logic

for test method updateTask_whenTaskDoesNotExist()

we have

verify(taskRepository.never()).save(any(Task.class));

never is an alias for times(0)

essential for verifying that save was not ever called

For INTEGRATION testing

entityManager.persistAndFlush(task);

to save into in memory db we use entityManager

persist makes it managed by JPA and flush helps to write to DB

calling persist doesn't guarantee that the sql stmt will be sent to db rightaway

flush() is called to update the db sync it and send ti db

so persist() -> arrange to put data in db